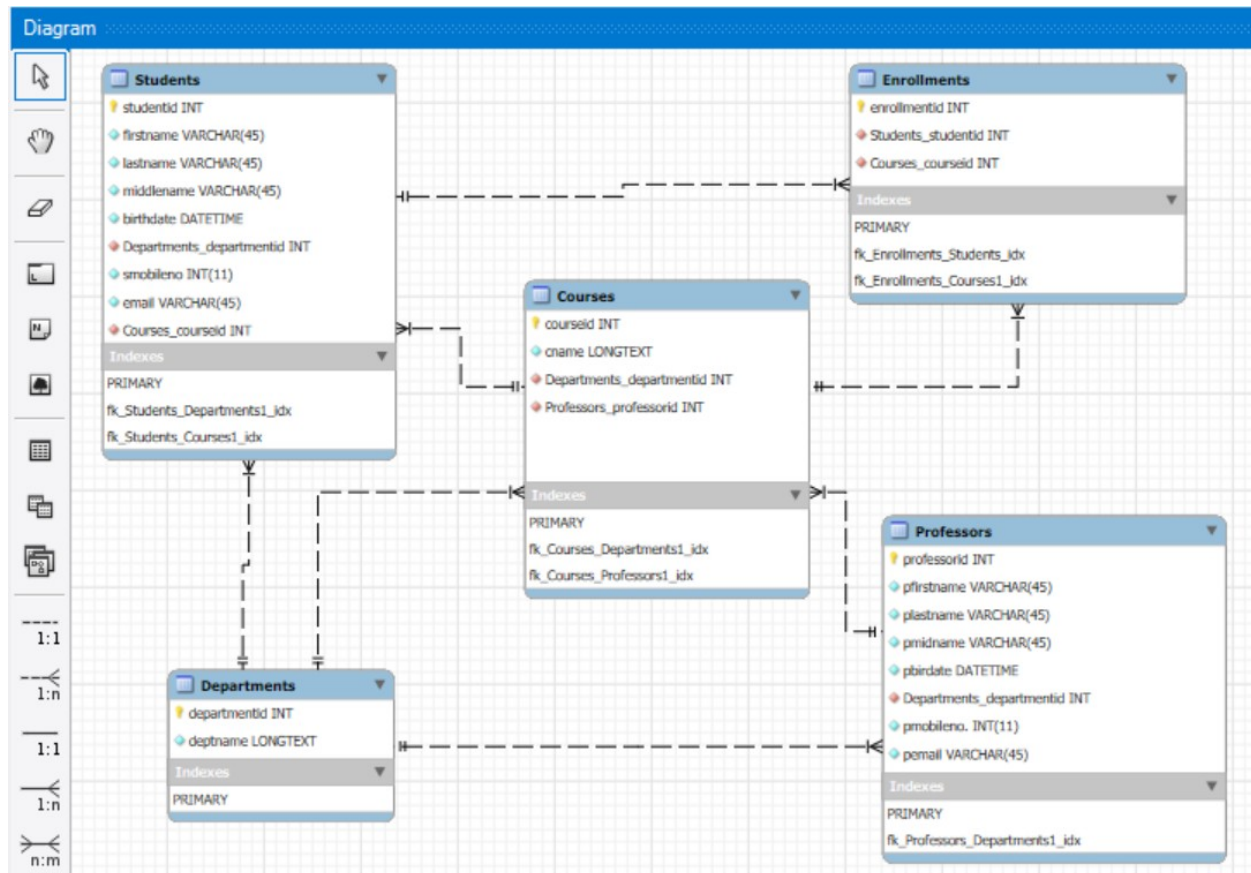


#ENROLLMENT SYSTEM DATABASE

##DATABASE SCHEMA



1. Students and Departments: Each student belongs to one department, but a department can have many students (**one-to-many**).
2. Students and Courses: A student can enroll in multiple courses, and each course can have many students (**many-to-many through the Enrollments table**).
3. Courses and Departments: Each course is offered by one department, but a department can offer multiple courses (**one-to-many**).
4. Courses and Professors: Each course is taught by one professor, but a professor can teach many courses (**one-to-many**).
5. Professors and Departments: Each professor belongs to one department, but a department can have many professors (**one-to-many**).

ENROLLMENT SYTEM

```
import sqlite3
from datetime import datetime
from tabulate import tabulate
```

```

# Database setup
connection = sqlite3.connect("enrollment_system.db")
cursor = connection.cursor()

# Create tables
cursor.execute('''
    CREATE TABLE IF NOT EXISTS departments (
        departmentid INTEGER PRIMARY KEY,
        deptname LONGTEXT NOT NULL
    );
''')

cursor.execute('''
    CREATE TABLE IF NOT EXISTS Professors (
        professorid INTEGER PRIMARY KEY,
        pfirstname VARCHAR(45) NOT NULL,
        plastname VARCHAR(45) NOT NULL,
        pmidname VARCHAR(45),
        pbirthdate DATETIME,
        Departments_departmentid INTEGER,
        pmobileno INT(11),
        pemail VARCHAR(45),
        FOREIGN KEY (Departments_departmentid) REFERENCES
Departments(departmentid)
    );
''')

cursor.execute('''
    CREATE TABLE IF NOT EXISTS Courses (
        courseid INTEGER PRIMARY KEY,
        cname VARCHAR(45) NOT NULL,
        Departments_departmentid INTEGER,
        Professors_professorid INTEGER,
        FOREIGN KEY (Departments_departmentid) REFERENCES
Departments(departmentid),
        FOREIGN KEY (Professors_professorid) REFERENCES
Professors(professorid)
    );
''')

cursor.execute('''
    CREATE TABLE IF NOT EXISTS Students (
        studentid INTEGER PRIMARY KEY,
        firstname VARCHAR(45) NOT NULL,
        lastname VARCHAR(45) NOT NULL,
        middlename VARCHAR(45),
        birthdate DATETIME,
        Departments_departmentid INTEGER,
        smobileno INT(11),
        email VARCHAR(45),

```

```

        Courses_courseid INTEGER,
        FOREIGN KEY (Departments_departmentid) REFERENCES
Departments(departmentid)
        FOREIGN KEY (Courses_courseid) REFERENCES
Courses(courseid)
    );
'''

cursor.execute('''
    CREATE TABLE IF NOT EXISTS Enrollments (
        enrollmentid INTEGER PRIMARY KEY AUTOINCREMENT,
        Students_studentid INTEGER,
        Courses_courseid INTEGER,
        FOREIGN KEY (Students_studentid) REFERENCES
Students(studentid),
        FOREIGN KEY (Courses_courseid) REFERENCES
Courses(courseid)
    );
'''

connection.commit()

# CLASS OF DIFFERENT TABLES

# Department class
class Department:
    def __init__(self, db, departmentid, deptname):
        self.db = db
        self.departmentid = departmentid
        self.deptname = deptname

    def save(self):
        if self.departmentid is not None:
            cursor.execute("INSERT INTO Departments (departmentid,
deptname) VALUES (?,?);", (self.departmentid, self.deptname))
            self.db.commit()

# Student class
class Student:
    def __init__(self, db, studentid, firstname, lastname, middlename,
birthdate, department_id, smobileno, email, courseid):
        self.db = db
        self.studentid = studentid
        self.firstname = firstname
        self.lastname = lastname
        self.middlename = middlename
        self.birthdate = birthdate
        self.department_id = department_id

```

```

        self.smobileno = smobileno
        self.email = email
        self.courseid = courseid

    def student_info(self):
        cursor.execute('''
            INSERT INTO Students (studentid, firstname, lastname,
            middlename, birthdate, Departments_departmentid, smobileno, email,
            Courses_courseid)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?);
        ''', (self.studentid, self.firstname, self.lastname,
        self.middlename, self.birthdate, self.department_id, self.smobileno,
        self.email, self.courseid))
        self.db.commit()

# Professor class
class Professor:
    def __init__(self, db, professorid, firstname, lastname,
    middlename, birthdate, department_id, pmobileno, email):
        self.db = db
        self.professorid = professorid
        self.firstname = firstname
        self.lastname = lastname
        self.middlename = middlename
        self.birthdate = birthdate
        self.department_id = department_id
        self.pmobileno = pmobileno
        self.email = email

    def name_prof(self):
        cursor.execute('''
            INSERT INTO Professors (professorid, pfirstname, plastname,
            pmidname, pbirthdate, Departments_departmentid, pmobileno, pemail)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?);
        ''', (self.professorid, self.firstname, self.lastname,
        self.middlename, self.birthdate, self.department_id, self.pmobileno,
        self.email))
        self.db.commit()

# Course class
class Course:
    def __init__(self, db, courseid, cname, department_id,
    professor_id):
        self.db = db
        self.courseid = courseid
        self.cname = cname
        self.department_id = department_id
        self.professor_id = professor_id

```

```

    def add_courses(self):
        cursor.execute('''
            INSERT INTO Courses (courseid, cname,
Departments_departmentid, Professors_professorid)
            VALUES (?, ?, ?, ?);
        ''', (self.courseid, self.cname, self.department_id,
self.professor_id))
        self.db.commit()

# Enrollment class
class Enrollment:
    def __init__(self, db, enrollmentid, student_id, course_id):
        self.db = db
        self.enrollmentid = enrollmentid
        self.student_id = student_id
        self.course_id = course_id

    def enrolled(self):
        cursor.execute('''
            INSERT INTO Enrollments (enrollmentid, Students_studentid,
Courses_courseid)
            VALUES (?, ?, ?);
        ''', (self.enrollmentid, self.student_id, self.course_id))
        self.db.commit()

connection.close()

```

##POPULATING EACH TABLES

```

# DEPARTMENT TABLES

if __name__ == "__main__":
    conn = sqlite3.connect("enrollment_system.db")
    cursor = conn.cursor()

    departments = [(1, 'College of Art and Science'),
        (2, 'College of Nursing'),
        (3, 'College of Business'),
        (4, 'College of Education')]

    for departmentid, deptname in departments:
        dept_add = Department(conn, departmentid, deptname)
        dept_add.save()
    print("Departments added successfully.")

# STUDENTS TABLE

if __name__ == "__main__":

```

```

conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

students = [(1001, 'John', 'Doe', 'A.', '2001-05-15', 1, 9876543210,
'john.doe@example.com', 10),
(1002, 'Jane', 'Smith', 'B.', '2002-03-22', 1, 9876543211,
'jane.smith@example.com', 9),
(1003, 'Adam', 'Johnson', 'C.', '2003-07-12', 2, 9876543212,
'adam.johnson@example.com', 8),
(1004, 'Eve', 'Taylor', 'D.', '2000-01-01', 2, 9876543213,
'eve.taylor@example.com', 7),
(1005, 'Liam', 'Brown', 'E.', '2001-09-10', 3, 9876543214,
'liam.brown@example.com', 6),
(1006, 'Emma', 'Wilson', 'F.', '2002-12-30', 3, 9876543215,
'emma.wilson@example.com', 5),
(1007, 'Noah', 'Lee', 'G.', '2000-04-18', 4, 9876543216,
'noah.lee@example.com', 4),
(1008, 'Sophia', 'White', 'H.', '2003-10-05', 4, 9876543217,
'sophia.white@example.com', 3),
(1009, 'Mason', 'Davis', 'I.', '2001-08-24', 1, 9876543218,
'mason.davis@example.com', 2),
(1010, 'Isabella', 'Martinez', 'J.', '2002-02-28', 1, 9876543219,
'isabella.martinez@example.com', 1),
(1011, 'James', 'Garcia', 'K.', '2003-11-11', 2, 9876543220,
'james.garcia@example.com', 10),
(1012, 'Charlotte', 'Lopez', 'L.', '2000-03-09', 2, 9876543221,
'charlotte.lopez@example.com', 9),
(1013, 'Logan', 'Hernandez', 'M.', '2001-06-06', 3, 9876543222,
'logan.hernandez@example.com', 8),
(1014, 'Amelia', 'Gonzalez', 'N.', '2002-08-20', 3, 9876543223,
'amelia.gonzalez@example.com', 7),
(1015, 'Lucas', 'Perez', 'O.', '2003-01-31', 4, 9876543224,
'lucas.perez@example.com', 6),
(1016, 'Mia', 'Ramirez', 'P.', '2001-12-15', 4, 9876543225,
'mia.ramirez@example.com', 5),
(1017, 'Benjamin', 'Lewis', 'Q.', '2002-07-07', 1, 9876543226,
'benjamin.lewis@example.com', 4),
(1018, 'Harper', 'Young', 'R.', '2003-09-19', 2, 9876543227,
'harper.young@example.com', 3),
(1019, 'Elijah', 'Allen', 'S.', '2000-05-27', 3, 9876543228,
'elijah.allen@example.com', 2),
(1020, 'Evelyn', 'King', 'T.', '2003-04-14', 4, 9876543229,
'evelyn.king@example.com', 1)]

for studentid, firstname, lastname, middlename, birthdate,
department_id, smobileno, email, courseid in students:
    student_add = Student(conn, studentid, firstname, lastname,
middlename, birthdate, department_id, smobileno, email, courseid)

```

```

        student_add.student_info()
print("Students added successfully.")

# COURSES TABLE

if __name__ == "__main__":
    conn = sqlite3.connect("enrollment_system.db")
    cursor = conn.cursor()

    courses = [(1, 'Information Technology', 1, 5001),
                (2, 'Business Administration', 3, 5003),
                (3, 'Nursing', 2, 5005),
                (4, 'Public Administration', 3, 5003),
                (5, 'Computer Science', 1, 5005),
                (6, 'Fine Arts', 1, 5004),
                (7, 'Biology', 4, 5004),
                (8, 'Secondary Education', 2, 5002),
                (9, 'Early Education', 2, 5002),
                (10, 'Data Science', 1, 5001)]

    for courseid, cname, department_id, professor_id in courses:
        courad = Course(conn, courseid, cname, department_id,
                        professor_id)
        courad.add_courses()
    print("Courses added successfully.")

#PROFESSORS TABLE

if __name__ == "__main__":
    conn = sqlite3.connect("enrollment_system.db")
    cursor = conn.cursor()

    professors = [(5001, 'Alice', 'Johnson', 'M.', '1975-04-12', 1,
                    9555627289, 'alice.johnson@example.com'),
                  (5002, 'Bob', 'Smith', 'A.', '1980-06-25', 2, 9874574582,
                    'bob.smith@example.com'),
                  (5003, 'Charlie', 'Brown', 'T.', '1985-09-10', 3, 9852123458,
                    'charlie.brown@example.com'),
                  (5004, 'Diana', 'Taylor', 'L.', '1978-03-15', 4, 9876654333,
                    'diana.taylor@example.com'),
                  (5005, 'Edward', 'King', 'R.', '1982-07-20', 1, 9876545644,
                    'edward.king@example.com')]

    for professorid, firstname, lastname, middlename, birthdate,
    department_id, smobileno, email in professors:
        prof = Professor(conn, professorid, firstname, lastname,
                        middlename, birthdate, department_id, smobileno, email)
        prof.name_prof()
    print("Professors added successfully.")

```

```

# ENROLLMENT TABLE

if __name__ == "__main__":
    conn = sqlite3.connect("enrollment_system.db")
    cursor = conn.cursor()

    enrollments = [(1, 1001, 1),
                    (2, 1002, 2),
                    (3, 1003, 3),
                    (4, 1004, 4),
                    (5, 1005, 5),
                    (6, 1006, 6),
                    (7, 1007, 7),
                    (8, 1008, 8),
                    (9, 1009, 9),
                    (10, 1010, 10),
                    (11, 1011, 10),
                    (12, 1012, 9),
                    (13, 1013, 8),
                    (14, 1014, 7),
                    (15, 1015, 6),
                    (16, 1016, 5),
                    (17, 1017, 4),
                    (18, 1018, 3),
                    (19, 1019, 2),
                    (20, 1020, 1)]

    for enrollmentid, student_id, course_id in enrollments:
        enrol = Enrollment(conn, enrollmentid, student_id, course_id)
        enrol.enrolled()
    print("Students are officially enrolled.")

```

CREATING, READING, UPDATING AND DELETING

```

# CREATING RECORDS
# Manual Entry of enrollment system:
import sqlite3
conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

# STUDENT DETAILS
studentid = input("Enter Student ID found in the enrollment form:")
firstname = input("Enter Student First Name:")
lastname = input("Enter Student Last Name:")
middlename = input("Enter Student Middle Name:")
birthdate = input("Enter Student Birth Date (YYYY-DD-MM):")

```



```

smobileno = input("Student Mobile Number(09XXXXXXXXX):")
email = input("Enter the Student Email:")
department_id = input("Enter the Department ID:")
courseid = input("Enter the Course ID:")

students = [(studentid, firstname, lastname, middlename, birthdate,
department_id, smobileno, email, courseid)]

for studentid, firstname, lastname, middlename, birthdate,
department_id, smobileno, email, courseid in students:
    student_add = Student(conn, studentid, firstname, lastname,
middlename, birthdate, department_id, smobileno, email, courseid)
    student_add.student_info()

print()
print(f"Student {lastname} with {studentid} Student ID is now Added to
the System.")
print()

# ENROLLMENT DETAILS
enrollmentid = input("Enter Enrollment ID:")
student_id = input("Enter Student ID:")
course_id = input("Enter Course ID:")

enrollments = [(enrollmentid, student_id, course_id)]

for enrollmentid, student_id, course_id in enrollments:
    enrol = Enrollment(conn, enrollmentid, student_id, course_id)
    enrol.enrolled()
print()
print(f"Student {lastname} with {studentid} Student ID is now
Officially Enrolled.")

# READING DATA IN THE TABLE
# Checking the values of each table
from tabulate import tabulate
conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

table2 = input("Please Enter the Table you want to view the values:")

query = f"SELECT * FROM {table2};"

cursor.execute(query)
tables2 = cursor.fetchall()

headers = [description[0] for description in cursor.description]

# Printing the result
print("Table Values:")

```

```

if tables2:
    print(tabulate(tables2, headers=headers, tablefmt="grid"))
else:
    print("No Data.")

# UPDATING DATA IN THE TABLE

conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

# Update a record
cursor.execute("""
    UPDATE Students
    SET smobileno = 9876543211, email = 'john.doe@example.com'
    WHERE studentid = 1001;
""")

## MODIFY is used when updating data type

print("Records has been successfully updated.")
conn.commit()

# DELETING DATA IN THE TABLE

conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

# Update a record
cursor.execute("""

    DELETE FROM Students
    WHERE studentid = 1021;
""")

print("Record was deleted. Database is updated.")
conn.commit()

```

##TABLE SCHEMA

```

#SHOWING ALL TABLES IN THE DATABASE
!pip install tabulate
from tabulate import tabulate
import sqlite3

# Connect to the database
conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
tables = cursor.fetchall()

```

```

headers = [description[0] for description in cursor.description]

# Print table names
print("Tables in the database:")
print(tabulate(tables, headers=headers, tablefmt="grid"))
# 5. Courses in a specific department

conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

table5 = input("Please Enter the Department Name:")

query = f'''SELECT
            c.courseid, c.cname
        FROM Courses c
        INNER JOIN Departments d ON c.Departments_departmentid
        = d.departmentid
        WHERE d.deptname = ?;
        '''

cursor.execute(query, (table5,))
tables5 = cursor.fetchall()

# Showing the Schema or Columns of the Table
conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

table1 = input("Please Enter the Table you want to view the Schema:")

cursor.execute(f"PRAGMA table_info({table1});")
design = cursor.fetchall()

print("Schema of the table:")
for column in design:
    print(column)

```

##"Query, serve as Stored Procedure"

```

# 1. SHOWING AND JOINING ALL THE TABLES
from tabulate import tabulate
import sqlite3

conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

query = """
        SELECT
            s.studentid, s.firstname, s.lastname, s.middlename,
s.birthdate, s.email,
            d.deptname AS student_department,

```

```

        c.courseid, c.cname AS course_name,
        p.pfirstname AS professor_firstname, p.plastname AS
professor_lastname
    FROM Students s
    INNER JOIN Departments d ON s.Departments_departmentid =
d.departmentid
    INNER JOIN Enrollments e ON s.studentid =
e.Students_studentid
    INNER JOIN Courses c ON e.Courses_courseid = c.courseid
    INNER JOIN Professors p ON c.Professors_professorid =
p.professorid
    """

cursor.execute(query)
results = cursor.fetchall()
headers = [description[0] for description in cursor.description] # get
the metadata of the description, and with the first element which is
the column name

# Print the results
print("Comprehensive Database Data:")
print(tabulate(results, headers=headers, tablefmt="grid"))

# 2. SHOWING STUDENT MAIN DETAILS

conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

query = """
        SELECT
            studentid,
            firstname,
            lastname,
            middlename,
            birthdate,
            strftime('%Y', 'now') - strftime('%Y', birthdate)
AS age
        FROM Students;
    """

cursor.execute(query)
tables = cursor.fetchall()
headers = [description[0] for description in cursor.description]

# Printing the result
print("Students Details:")
print(tabulate(tables, headers=headers, tablefmt="grid"))

# 3. SHOWS THE STUDENTS IN AN SPECIFIC COURSE

conn = sqlite3.connect("enrollment_system.db")

```

```

cursor = conn.cursor()

table3 = input("Please Enter the Course you want to view the
Student/s:")

query = f"""
        SELECT
            s.studentid, s.firstname, s.lastname,
s.middlename, s.email
        FROM Students s
        INNER JOIN Enrollments e ON s.studentid =
e.Students_studentid
        INNER JOIN Courses c ON e.Courses_courseid =
c.courseid
        WHERE c.cname = ?;
    """

cursor.execute(query, (table3,))
tables3 = cursor.fetchall()
headers = [description[0] for description in cursor.description]

# Printing the result
print(f"Students enrolled in the course of {table3}:")
print(tabulate(tables3, headers=headers, tablefmt="grid"))

# 4. Courses taught by a specific professor

conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

table4 = input("Please Enter your professor's last name:")
query = """
        SELECT
            c.courseid, c.cname,
            p.pfirstname, p.plastname
        FROM Courses c
        INNER JOIN Professors p ON c.Professors_professorid =
p.professorid
        WHERE p.plastname = ?;
    """

cursor.execute(query, (table4,))
tables4 = cursor.fetchall()
headers = [description[0] for description in cursor.description]

# Printing the result
print(f"Courses taught by Mr./Ms. {table4}:")
print(tabulate(tables4, headers=headers, tablefmt="grid"))

# 5. Courses in a specific department

```

```
conn = sqlite3.connect("enrollment_system.db")
cursor = conn.cursor()

table5 = input("Please Enter the Department Name:")

query = f'''SELECT
            c.courseid, c.cname
        FROM Courses c
        INNER JOIN Departments d ON c.Departments_departmentid
        = d.departmentid
        WHERE d.deptname = ?;
        '''

cursor.execute(query, (table5,))
tables5 = cursor.fetchall()

# Printing the result
print(f"Courses in Department {table5}:")
print(tabulate(tables5, headers=headers, tablefmt="grid"))

connection.close()
```